Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory work work #2.1

Sequential Bare-Metal Systems. Scheduler. Cooperative Multitasking.

During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur Cravtov
std. gr. FAF-231

Verified by:
Alexei Martîniuc
asist. univ

Chișinău, 2026

Contents

1 Objective and Technical Analysis **2**
1.1 Bare-metal Sequential Systems and Cooperative Multitasking 2
1.2 System Tick and Hardware Timer Configuration 2
1.3 Interrupt Service Routine (ISR) and Scheduler Update 3
1.4 Timing Control: Recurrence and Offset 4

2 System Design **4**
2.1 System Architecture 4
2.2 Flowchart of the program 5
2.2.1 Individual Task Flowcharts 5
2.2.2 General System Flowchart 8

3 Implementation and Results **9**
3.1 Software Layering 9
3.2 Task-Specific Implementation Details 9
3.2.1 Reading and Measurement 9
3.2.2 Global Statistics Management 9
3.2.3 Periodic Reporting 9
3.3 Hardware Required 9
3.4 Layered Design (Hardware-Software) 10
3.4.1 Button driver 10
3.5 Electrical Schematics 12
3.6 Simulation 13
3.7 Physical testing 16
3.8 Challanges 17

4 Conclusion **17**

References **18**

A Source Code **18**

1 Objective and Technical Analysis

The objective of this laboratory work is to study operating systems for embedded systems by implementing a multitasking monitor system on a bare-metal microcontroller. The system will manage multiple concurrent tasks, including button state monitoring, statistics calculation, LED signaling, and periodic reporting via UART. The implementation will utilize a cooperative scheduling approach, where tasks yield control voluntarily, and timing is managed through a hardware timer interrupt.

Through this work, the following learning outcomes are pursued:

- Understanding the foundations of non-preemptive scheduling and task management.
- Implementing a lightweight task scheduler based on hardware timer interrupts.
- Mastering the concepts of task recurrence and initial offset to optimize CPU load distribution.
- Developing a modular software architecture (MCAL/ECAL/APP) to ensure code portability and maintainability.
- Handling inter-task communication using shared global state and change-detection flags.

1.1 Bare-metal Sequential Systems and Cooperative Multitasking

In the realm of embedded systems, a bare-metal sequential system operates without a formal kernel, instead using a "super-loop" or a cooperative scheduler to manage task execution [6]. The defining characteristic of this approach is that tasks are *non-preemptive*; once a task starts, it runs to completion (or its current iteration's end) before yielding control. This eliminates the need for complex synchronization primitives like mutexes (since only one task runs at a time), but it requires the developer to ensure that no task "blocks" the CPU for too long. A "spin-lock" or a long `delay()` in one task will stall the entire system.

1.2 System Tick and Hardware Timer Configuration

The foundation of our cooperative scheduler is a periodic "system tick" generated by a hardware timer. In this implementation, we utilize **Timer1** of the ATmega microcontroller, configured in Clear Timer on Compare Match (CTC) mode [3]. This allows for precise, periodic interrupts without the drift associated with software-only delays.

The configuration involves the following registers:

- **TCCR1A / TCCR1B**: Timer/Counter Control Registers. We set `WGM12` to enable CTC mode and `CS11` to set the prescaler to 8 [4].
- **OCR1A**: Output Compare Register A. This defines the "target" count for the timer. The value is calculated as [3]:

$$OCR1A = \frac{F_{CPU}}{Prescaler \times TargetFrequency} - 1$$

  For a 16MHz clock (F_{CPU}), a prescaler of 8, and a target frequency of 1000Hz (1ms), the value is `1999`.
- **TIMSK1**: Timer Interrupt Mask Register. Setting `OCIE1A` enables the interrupt when the timer matches the `OCR1A` value.

1.3 Interrupt Service Routine (ISR) and Scheduler Update

When Timer1 reaches the value in `OCR1A`, the hardware triggers a "Compare Match" interrupt, executing the associated **Interrupt Service Routine (ISR)** [5]. Inside the `ISR(TIMER1_COMPA_vect)`, we call `Scheduler::update()`, which decrements the counters for all registered tasks.

Because the ISR is triggered by hardware, it provides a highly stable timebase. However, to maintain system stability, the ISR must be extremely brief. Our implementation follows this rule by only performing simple decrements in the ISR, while the actual task execution happens in the main `loop()` context. This ensures that the system tick is never missed and that higher-priority hardware events can still be serviced.

The high level flow of the scheduler is as follows:

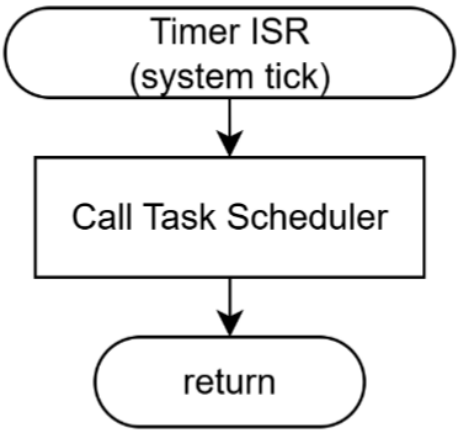


Figure 1: Scheduler Flow: Timer ISR

1.4 Timing Control: Recurrence and Offset

Effective multitasking requires precise control over when and how often tasks are executed:

- **Recurrence (Period)**: Defines the interval (in ticks) between successive executions.
- **Offset**: Defines the initial delay before the first execution, allowing tasks to be staggered and preventing CPU "spikes" where multiple heavy tasks attempt to run on the same tick.
- **Recurrence Control**: Represents the current countdown until the next execution. It is initialized to `offset` and reset to `recurrence` after each execution.

2 System Design

2.1 System Architecture

Figure 2 illustrates the overall architecture of the multitasking monitor system. The core components include:

- **Microcontroller (ATmega)**: The central processing unit that executes the tasks and manages hardware interactions.
- **Button**: A digital input device used to capture user interactions. It is connected with an internal pull-down resistor, allowing for simple state detection (pressed vs. released).
- **LEDs**: Three LEDs (Green, Red, Yellow) provide visual feedback. The Green and Red LEDs indicate the type of last button press (short or long), while the Yellow LED blinks to signal that new statistics are available.
- **UART Interface**: Used for serial communication to output statistics to the STDIO interface. Configured at a high baud rate (1,000,000 baud) to minimize the time spent in serial transmission and prevent blocking the scheduler.

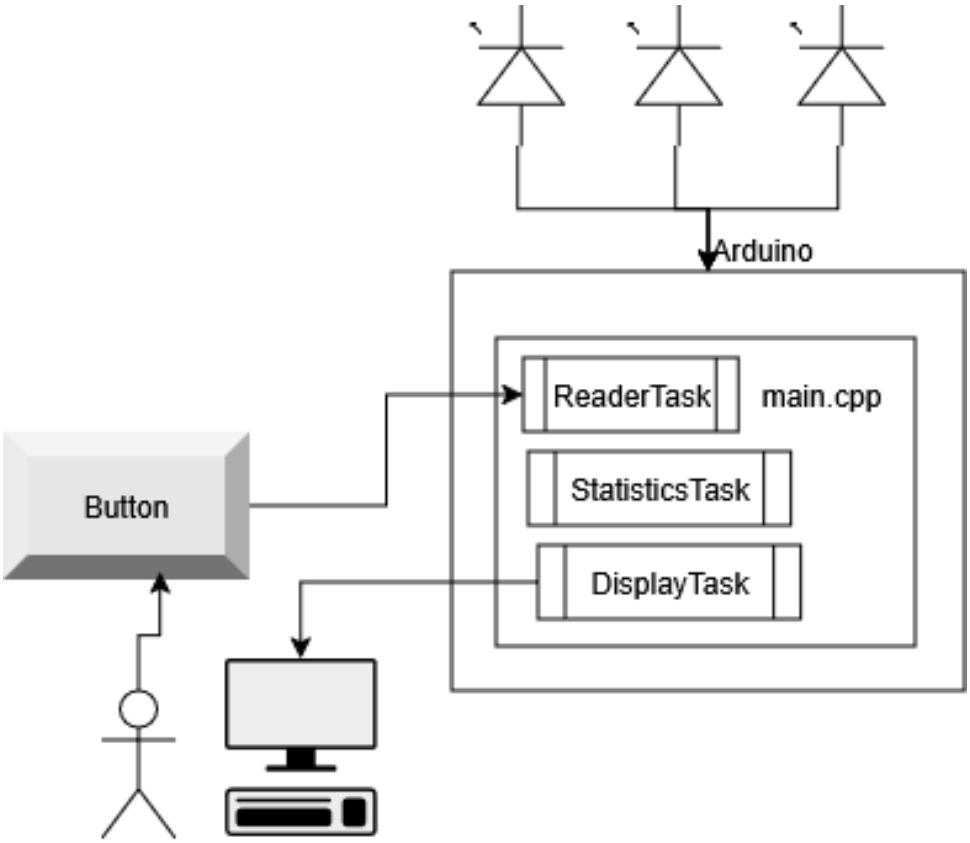


Figure 2: System Architecture Diagram

2.2 Flowchart of the program

2.2.1 Individual Task Flowcharts

Figure 3 depicts the flow of the Reading Task, which is responsible for monitoring the button state and measuring press durations. The task samples the button every 2ms, updating its internal state accordingly. When a button release is detected, it calculates the duration of the press and updates the shared "mailbox" variables for the Statistics Task to process.

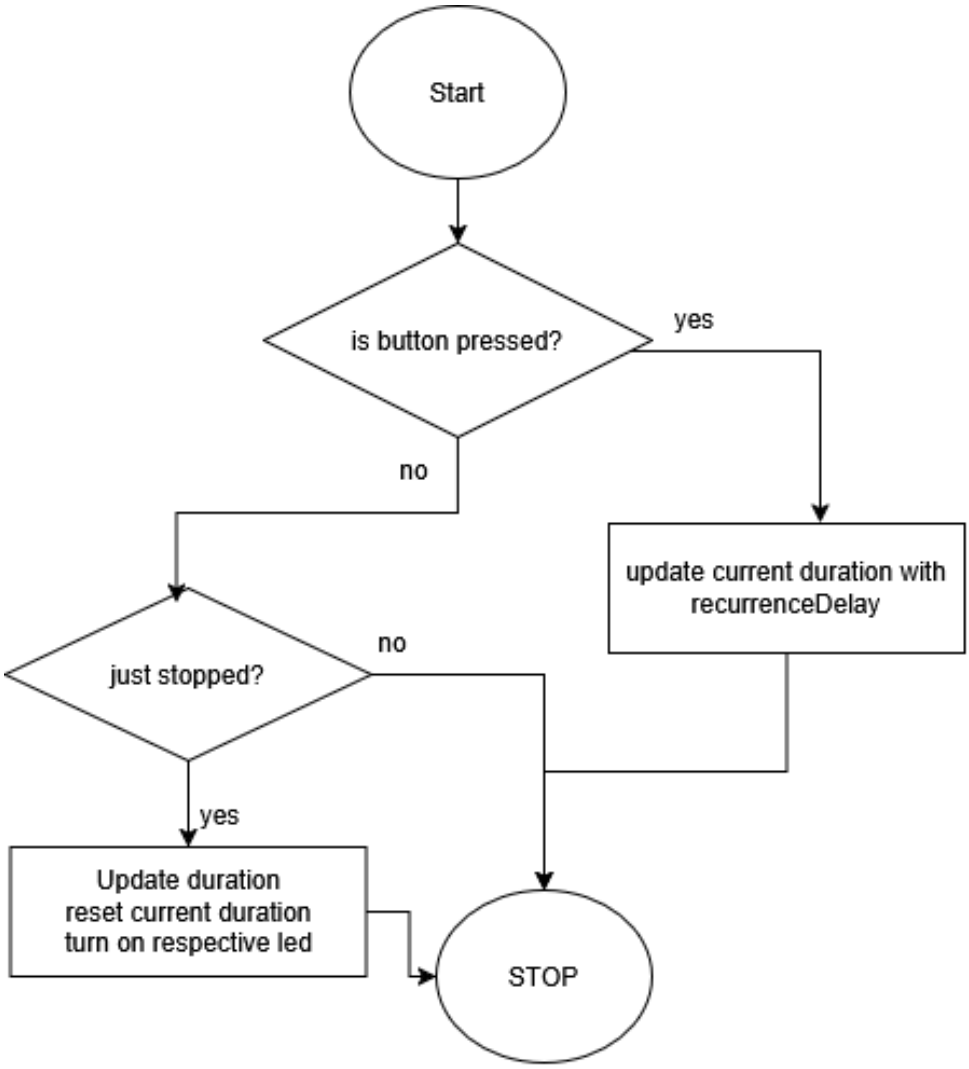


Figure 3: Reading Task Flowchart

Figure 4 illustrates the flow of the Statistics Task. It continuously checks for updates from the Reading Task by comparing timestamps. When a new press duration is available, it updates the total count and cumulative duration, which are used for calculating averages. It also toggles the Yellow LED to signal the length of the last press (short or long) and to indicate that new statistics are ready for display.

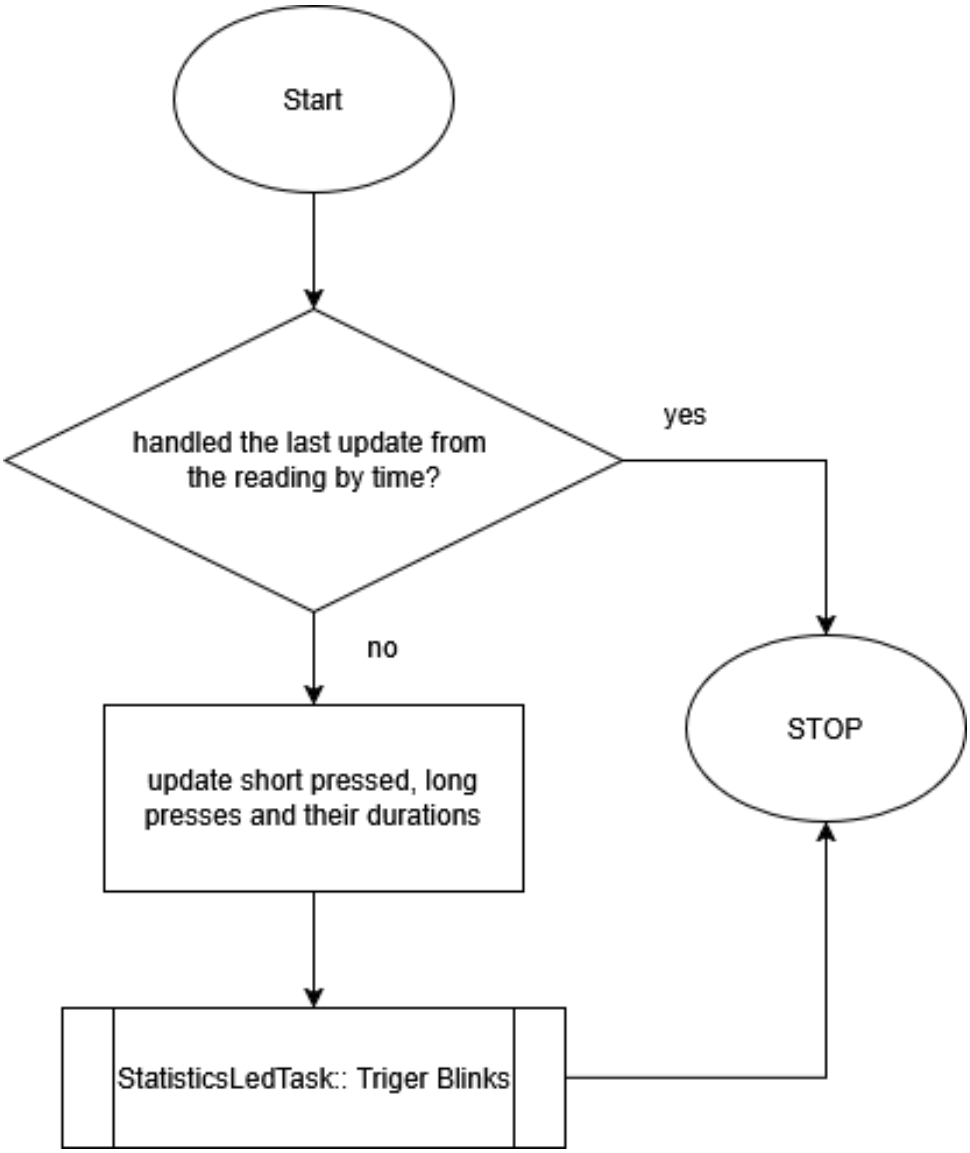


Figure 4: Statistics Task Flowchart

Figure 5 shows the flow of the Display Task, which runs every 10 seconds. It retrieves the current statistics, calculates the average press duration, and outputs the results via UART.

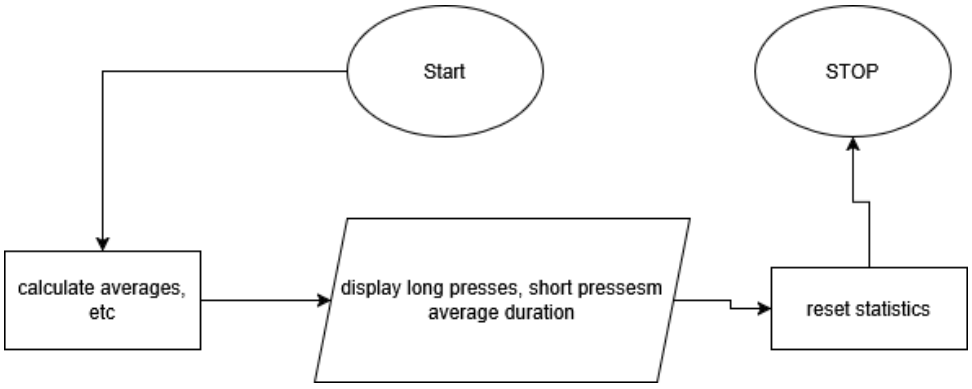


Figure 5: Display Task Flowchart

2.2.2 General System Flowchart

Figure 6 provides an overview of the entire system's flow. It shows how the tasks interact with each other and with the hardware components, highlighting the cooperative multitasking nature of the design. The timer ISR ensures that tasks are executed at their specified intervals, while the main loop continuously checks for task readiness and executes them accordingly.

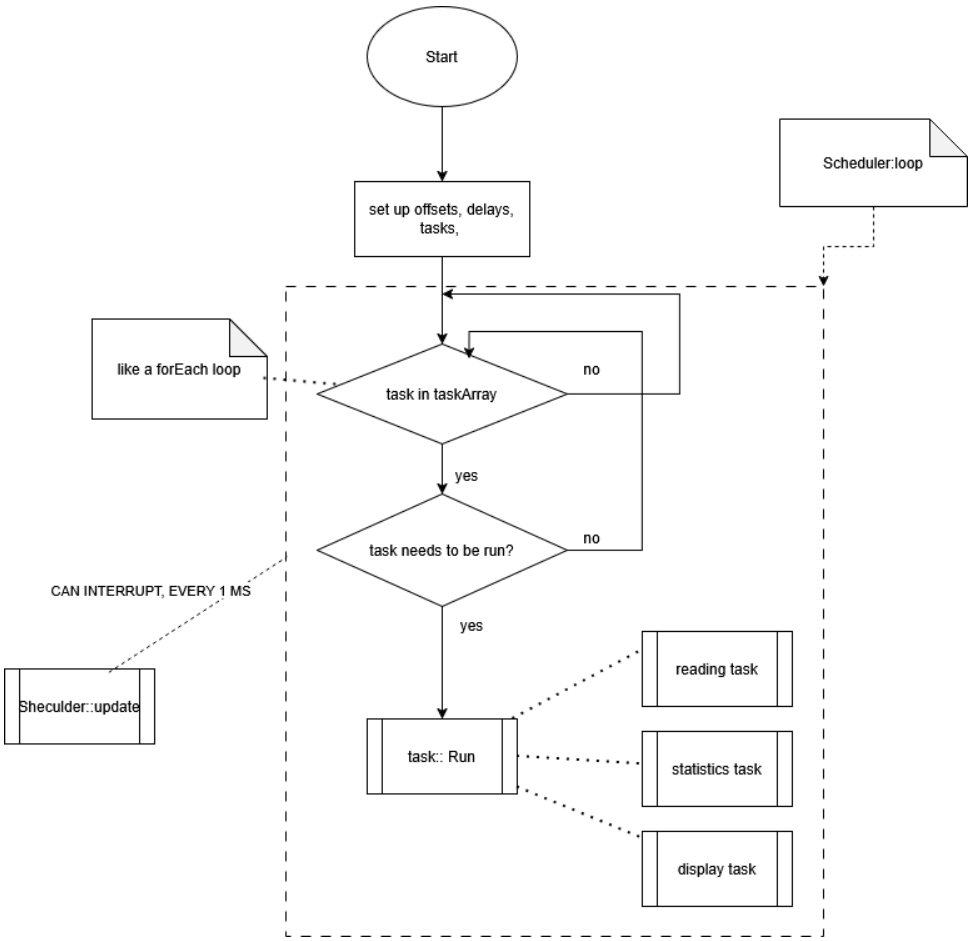


Figure 6: General System Flowchart

3 Implementation and Results

3.1 Software Layering

The code is organized into a clean directory structure. The `Scheduler` module provides the core timing infrastructure, while the `src/labs/lab2_1/src/` directory contains the specific task implementations. This structure allows the same Scheduler to be reused for different projects by simply changing the task registration in `main.cpp`.

3.2 Task-Specific Implementation Details

3.2.1 Reading and Measurement

The `ReadingTask.cpp` handles debouncing implicitly by its 2ms sampling rate. It stores the `currentDuration` as long as the button is active. Once the button is released, the duration is "published" to the `ReadingTask::lastDuration` variable, and `ReadingTask::lastUpdatedTime` is set to the current `millis()` count. This acting as a "mailbox" for the Statistics Task.

3.2.2 Global Statistics Management

The `StatisticsTask.cpp` maintains the running totals. By comparing its own `lastHandledUpdateTime` with the Reading Task's timestamp, it ensures that each press is processed exactly once even if the Statistics Task runs more frequently than the presses occur.

3.2.3 Periodic Reporting

The `DisplayTask.cpp` performs a "snapshot" of the data every 10 seconds. It calculates the average press duration using integer arithmetic to avoid the overhead of a floating-point library, formatting the results into seconds and hundredths of seconds (e.g., "1.25s").

3.3 Hardware Required

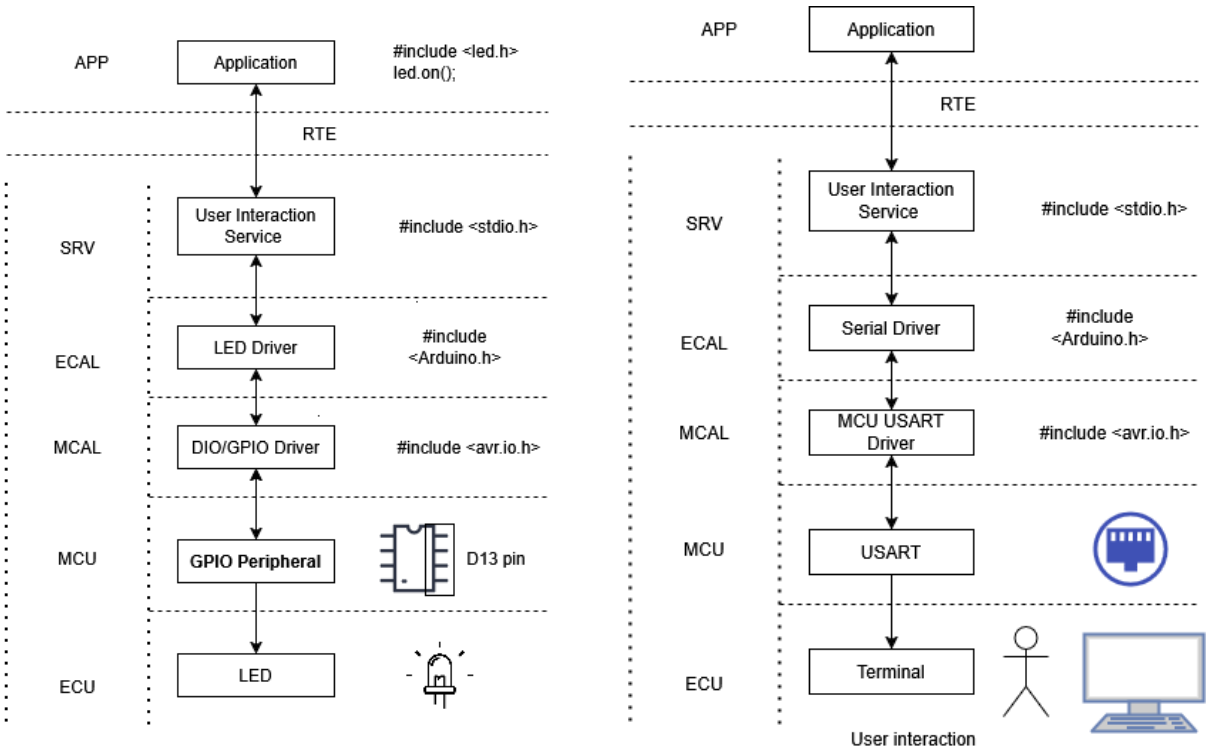
The implementation was tested on an ATmega-based development board (e.g., Arduino Uno) with the following hardware components:

- **Button**: Connected to a digital input pin with `INPUT` mode and external resistor.
- **LEDs**: Three LEDs connected to digital output pins with appropriate current-limiting resistors.

- **UART Interface**: The onboard USB-to-serial converter is used for communication with the host computer.

3.4 Layered Design (Hardware-Software)

This project reuses the last laboratory works hardware such as Led, Serial, so they are not repeated in this report, but avaliable in report of lab 1.1 and 1.2 [1, 2]. The diagrams themselves are the following:



(a) LED layering (MCAL/ECAL/APP)

(b) Serial layering (MCAL/ECAL/APP)

Figure 7: Module layering examples for LED and Serial drivers

3.4.1 Button driver

Button driver is new for this lab, and it is implemented in a similar layered manner, represented in figure 8

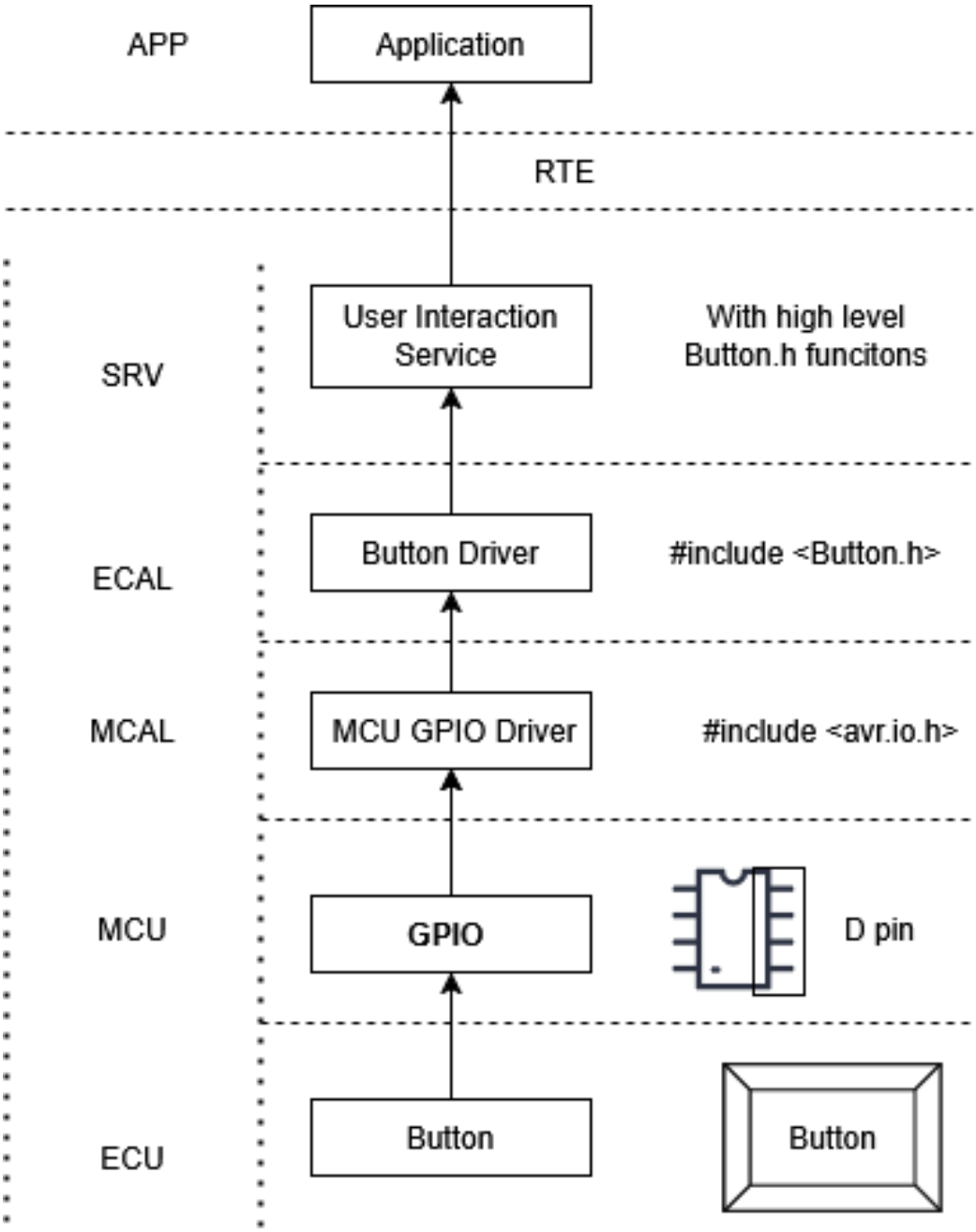


Figure 8: Button layering

The Button driver abstract the hardware details of reading the button state, providing a simple interface for the Reading Task to query the current state. This modular design allows for easy replacement or enhancement of the button handling logic (e.g., adding software debouncing) without affecting the rest of the system.

The layers are the following:

1. **APP** – Application Layer: Calls the service to check button state and reacts accordingly (e.g., trigger an event or update display).

2. **RTE** – Runtime Environment: Routes communication between the application and

the service layer.

3. **SRV** – Service Layer: Contains the User Interaction Service, handles button press detection, debouncing, and state change logic.

4. **ECAL** – ECU Abstraction Layer: Contains the Button Driver (`Button_Driver.h/.c`), provides a `button_read()` function wrapping the GPIO read.

5. **MCAL** – Microcontroller Abstraction Layer: MCU GPIO Driver configures the button pin as `INPUT` using `avr/io.h`, reads pin state via `DDRD` / `PIND` registers.

6. **MCU** – Microcontroller Unit: Physical GPIO pin (e.g., D2 or D7) configured as input, with a pull down resistor for the button connection.

7. **ECU** – Electronic Control Unit: The actual physical push button hardware connected to the MCU pin.

3.5 Electrical Schematics

Electronical circuit of the system is represented in figure 9. It consists of a single button connected to a digital input pin with an external pull down resistor, three LEDs connected to digital output pins with current-limiting resistors, and the UART interface for serial communication. The button is configured to pull the input pin high when pressed, allowing for simple state detection in the Reading Task.

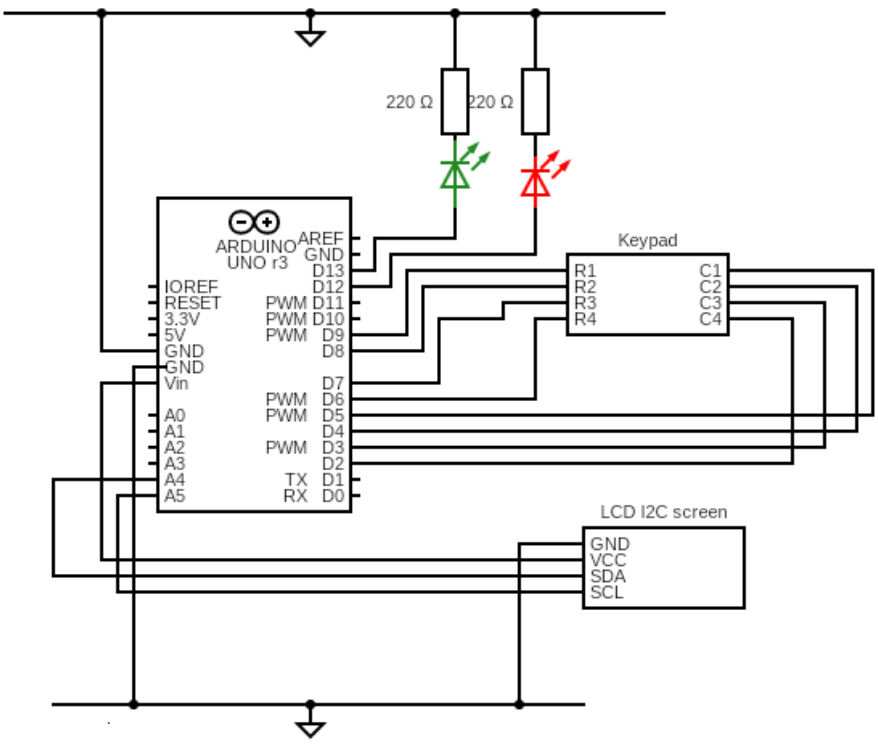


Figure 9: Electrical Schematic of the Multitasking Monitor System

3.6 Simulation

The Simulation of the system will be performed using Wokwi.

A short press followed by a release is shown in figure 10. The Green LED turns on to indicate a short press, and the Yellow LED blinks to signal that new statistics are available. The serial output would show the updated statistics after the next 10-second reporting interval.

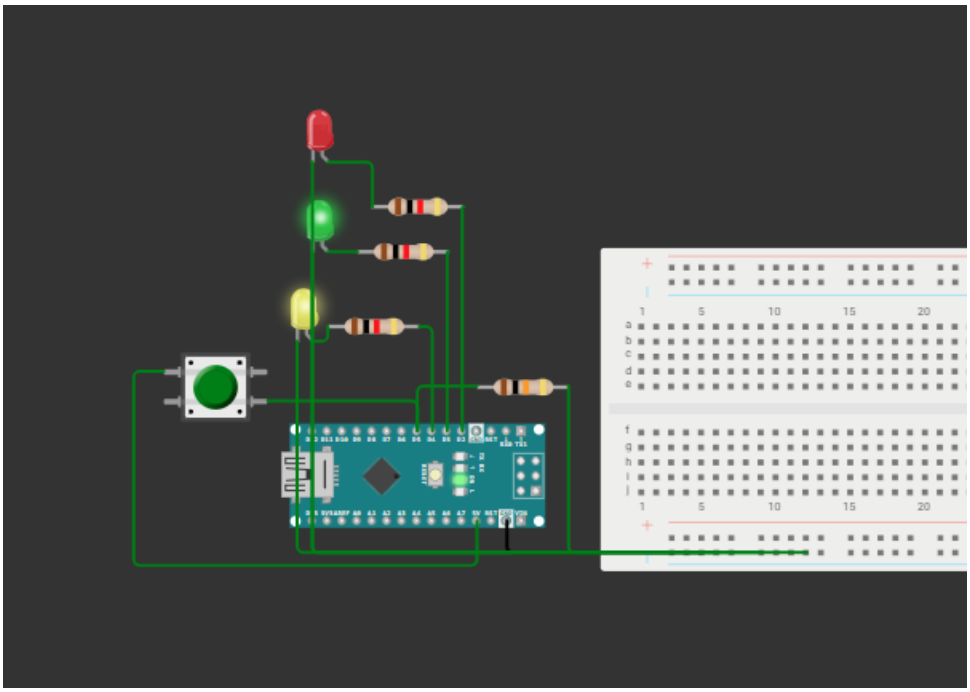


Figure 10: Simulation of a short button press (0.5s)

A long press followed by a release is shown in figure 11. The Red LED turns on to indicate a long press, and the Yellow LED blinks to signal that new statistics are available. The serial output would show the updated statistics after the next 10-second reporting interval. Moreover, the serial output would reflect the increased average press duration due to the long press, demonstrating that the system correctly distinguishes between short and long presses and updates its statistics accordingly.

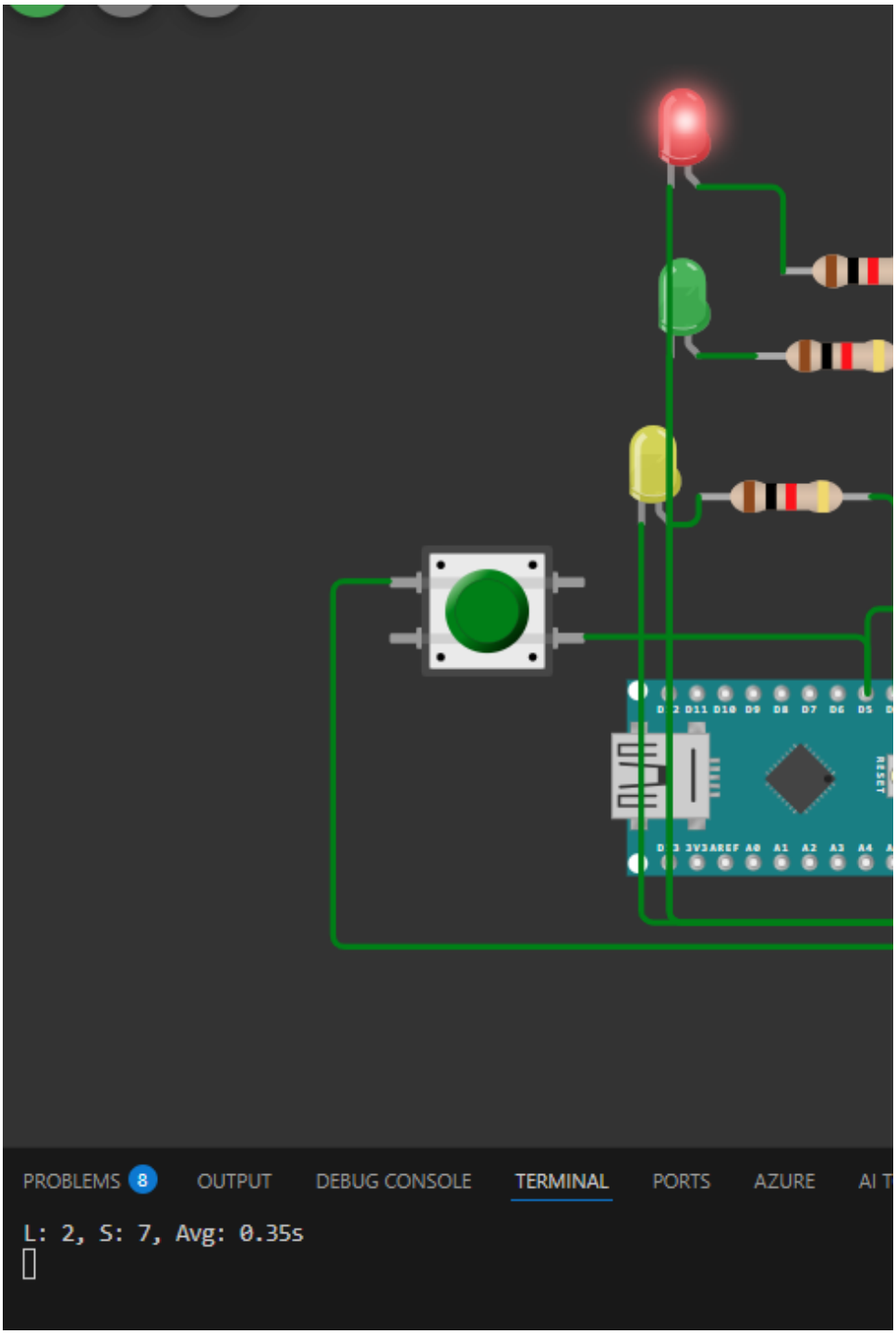


Figure 11: Simulation of a long button press (1.5s)

Next, the statistics is indeed reset after 10 seconds, as shown in figure 12. The Yellow LED blinks to indicate that new statistics are available, and the serial output would show the reset statistics (e.g., total presses: 0, average duration: 0s) after the next 10-second reporting interval.

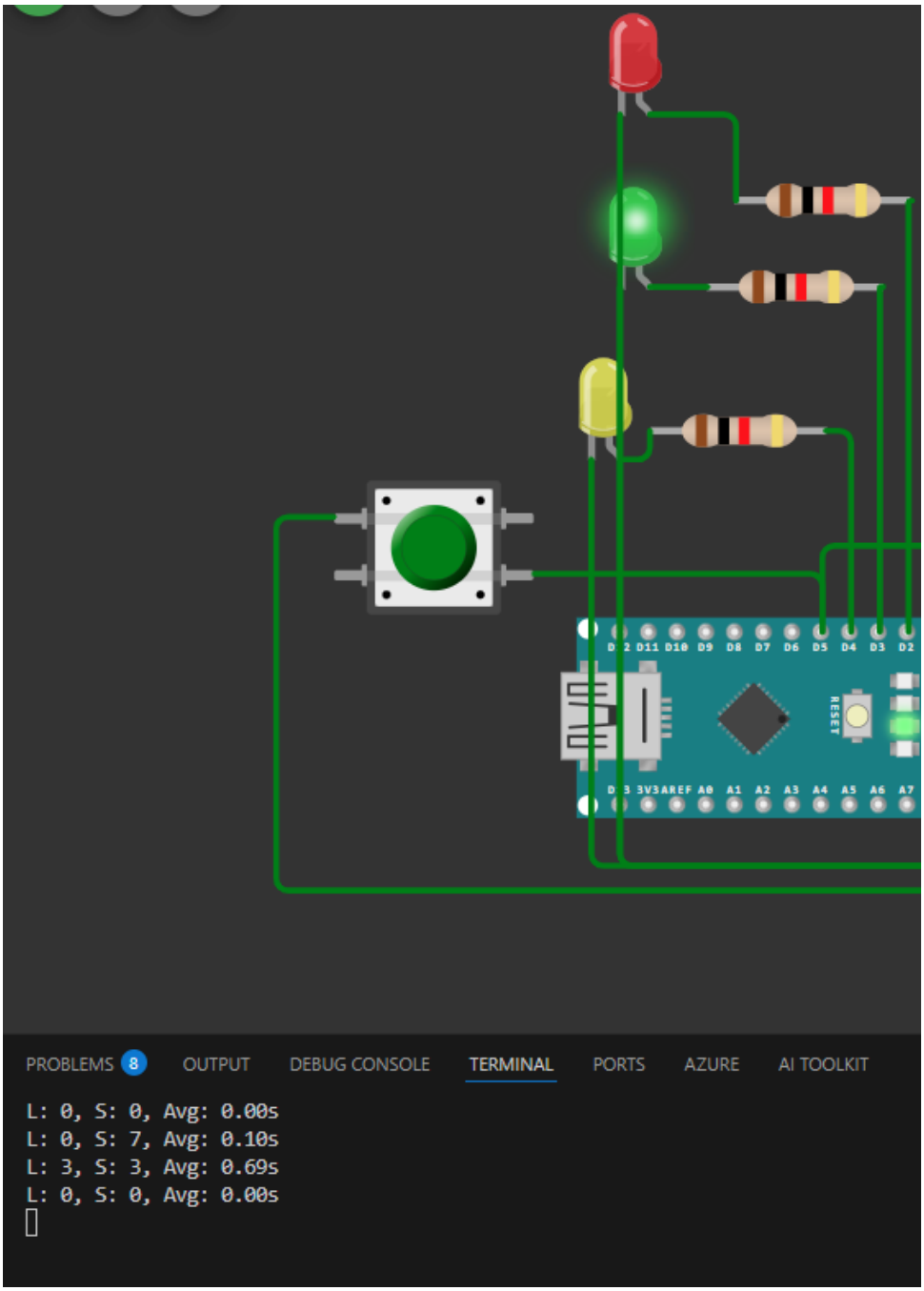


Figure 12: Simulation of statistics reset after 10 seconds

3.7 Physical testing

The real testing happens on Nano board, presented on figure 13. The behavior observed in the physical testing matches the simulation results, confirming that the system correctly handles button presses, updates statistics, and reports via UART as designed.

It works as expected and demonstrates the effectiveness of the cooperative multitasking approach in a bare-metal embedded system. The LEDs provide immediate visual feedback, while the serial output allows for detailed monitoring of the system's perfor-

mance and statistics over time.

3.8 Challanges

During this laboratory work, one major challange was addressed: the system stopped working as expected after a minute or so. The initial though was that a variable overflow might be happening. With futher inverstigation, it was indded the cause in `StatisticsTask::lastHandledUpdateTime` variable, which was defined as `uint16_t`. Since the `millis()` function returns an `unsigned long` (32-bit), the 16-bit variable would overflow after approximately 65 seconds, causing the timestamp comparison logic to fail and preventing the Statistics Task from processing new updates. By changing the type of `lastHandledUpdateTime` to `unsigned long`, we ensured that it can correctly handle the full range of values returned by `millis()`, thus resolving the issue and allowing the system to operate correctly over extended periods.

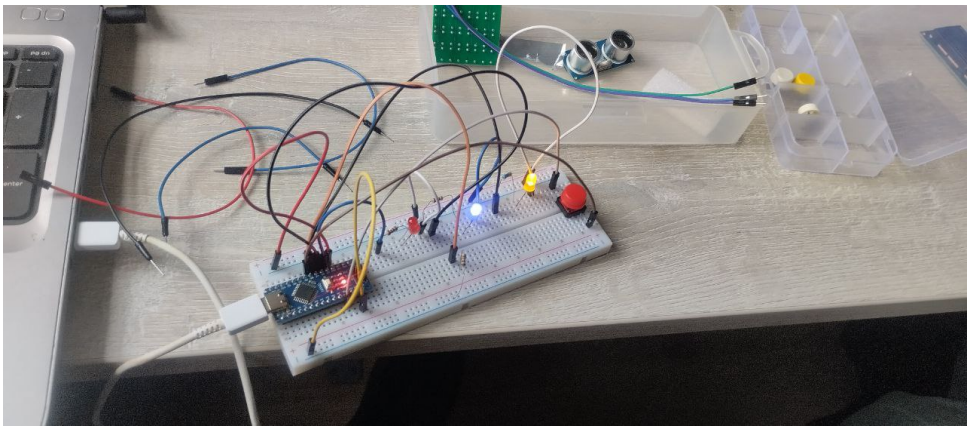


Figure 13: Physical Testing of the Multitasking Monitor System

4 Conclusion

This laboratory work successfully demonstrated the viability and efficiency of a modular bare-metal multitasking architecture. By implementing a custom task scheduler, we were able to manage concurrent operations—ranging from high-frequency button polling (2ms) to long-period serial reporting (10,000ms)—on a single-core microcontroller without the need for a complex RTOS kernel.

Key conclusions include:

- **Efficiency**: The use of recurrence and offset staggering effectively balanced the CPU load, maintaining system responsiveness even during serial reporting.
- **Modularity**: The task-based approach significantly simplified the development

process, allowing each feature (reading, stats, signaling, reporting) to be implemented and debugged in isolation.

- **Scalability**: This bare-metal pattern provides a solid foundation for more advanced projects, offering a clear upgrade path toward preemptive RTOS systems while keeping the application logic largely hardware-agnostic.

In summary, the implementation met all technical requirements, providing a robust solution for a multitasking monitor system and reinforcing the importance of methodical software design in embedded engineering.

References

[1] Lab 1.1 report with Led module implementation and explanation. `https://github.com/TimurCravtov/EmbeddedSystemsLabs/blob/main/reports/l1_1/FAF-231_SI_LAB1-1_Timur-Cravtov.pdf`

[2] Lab 1.2 report with Serial module implementation and explanation. `https://github.com/TimurCravtov/EmbeddedSystemsLabs/blob/main/reports/l1_2/FAF-231_SI_LAB1-2_Timur-Cravtov.pdf`

[3] Arduino Slovakia, *Timer in CTC Mode Tutorial*, `https://www.arduinoslovakia.eu/blog/2017/6/timer-v-rezime-ctc?lang=en`

[4] Nick Gammon, *Microprocessors: Timers and Counters*, `http://www.gammon.com.au/timers`

[5] Oscar Liang, *Arduino Timer and Interrupt Tutorial*, `https://oscarliang.com/arduino-timer-and-interrupt-tutorial/`

[6] Arduino Forum, *Simple Multitasking for Arduino*, `https://forum.arduino.cc/t/simple-multitasking-for-arduino/350920`

[7] SEI Indrumar Lab 3, Lab 3.1: Sisteme secvențiale (bare-metal) requirements.

[8] Shared Code Library - Timur Cravtov `https://github.com/TimurCravtov/EmbeddedSystemsLabs`

A Source Code

Besides this appendix, the full source code is available in the GitHub repository [8].

`main.cpp`

```cpp
#include <Arduino.h>
#include <stdio.h>

#include <button/button.h>
#include <led/led.h>
#include <serialio/serialio.h>

#include "DisplayTask.h"
#include "ReadingTask.h"
#include "Scheduler.h"
#include "StatisticsTask.h"
#include "Timer.h"


// leds
constexpr uint8_t redLedPin = 2;
constexpr uint8_t greenLedPin = 3;
constexpr uint8_t yellowPin = 4;

// button
constexpr uint8_t buttonPin = 5;

// led objects
Led redLed(redLedPin);
Led greenLed(greenLedPin);
Led yellowLed(yellowPin);
Button button(buttonPin);

// task config
namespace ReadingTask { uint16_t recurrenceDelay = 2; }
namespace StatisticsTask { uint16_t recurrenceDelay = 2; }
namespace DisplayTask { uint16_t recurrenceDelay = 10000; }

// offsets
namespace ReadingTask { uint16_t offset = 3; }
namespace StatisticsTask { uint16_t offset = 2; }
namespace DisplayTask { uint16_t offset = 10000; }

```

```
// add tasks
void setupScheduler() {
  Scheduler::addTask(StatisticsTask::run, StatisticsTask::recurrenceDelay,
                     StatisticsTask::offset);
  Scheduler::addTask(ReadingTask::run, ReadingTask::recurrenceDelay,
                     ReadingTask::offset);
  Scheduler::addTask(DisplayTask::run, DisplayTask::recurrenceDelay,
                     DisplayTask::offset);
  Scheduler::addTask(StatisticsLedTask::run, StatisticsLedTask::recurrenceDelay,
                     StatisticsLedTask::offset);
}

void setup() {
  redLed.init();
  greenLed.init();
  yellowLed.init();
  button.init();
  Timer::init();

  setupScheduler();

  Serial.begin(9600);
  redirectSerialToStdio(true, true, true);
}

void loop() {
    Scheduler::loop();
}
```

Listing 1: main.cpp (lab2_1)

`Scheduler.h`

```
#pragma once

#include <stdint.h>
#include "Task.h"
```

```

/// @brief A simple cooperative scheduler for managing tasks in an embedded system. It allows you to add tasks with specific recurrence delays and offsets, and it executes them in a loop based on their configured timing.
namespace Scheduler {
    /// @brief An array of TaskConfig with dynamic size. Used internally by the implentation
    extern TaskConfig* tasks;

    /// @brief The number of tasks currently registered in the scheduler. Used internally by the implementation.
    extern uint8_t taskCount;

    /// @brief Adds a new task to the scheduler with the specified function, recurrence delay, and offset. The task will be executed based on its timing configuration.
    void addTask(void (*taskFunction)(), uint16_t recurrenceDelay, uint16_t offset);

    /// @brief This will be called in the main loop to execute the tasks based on their timing. It checks each task's recurrence control and executes the task function when the control reaches zero, then resets the control to the task's recurrence delay.
    void loop();

    /// @brief This function is called on each system tick (e.g., from a timer interrupt) to update the scheduler's internal timing and determine when tasks should be executed. It decrements the recurrence control for each task and triggers task execution when the control reaches zero.
    void update();

    /// @brief Registers a task with the given configuration. This function is used internally to set up the task's function pointer, recurrence delay, offset, and initial
```

```
        recurrence control based on the offset.
    /// @param config A pointer to the TaskConfig structure that will be filled with the task's configuration.
    /// @param taskFunction A pointer to the function that implements the task's behavior.
    /// @param recurrenceDelay The delay in ticks between each execution of the task after the initial offset.
    /// @param offset The initial delay in ticks before the task is first executed.
    void registerTask(TaskConfig *config, void (*taskFunction)(), uint16_t recurrenceDelay, uint16_t offset);
}
```

Listing 2: Scheduler.h

`ReadingTask.cpp`

```
#include "ReadingTask.h"
#include <button/button.h>
#include <led/led.h>
#include <stdint.h>

void visualizeButtonPressDuration();

namespace ReadingTask {
    extern uint16_t recurrenceDelay;
    extern uint16_t offset;
    unsigned long lastUpdatedTime = -1;
    unsigned long lastDuration = 0;
}

extern Button button;

unsigned long currentDuration = 0;

void ReadingTask::run() {

    bool isPressed = button.isPressed();
    bool itsFreshPress = currentDuration == 0;

    bool pressJustStopped = !isPressed && !itsFreshPress;
```

```

    if (pressJustStopped) {
        lastUpdatedTime = millis();
        ReadingTask::lastDuration = currentDuration;
        currentDuration = 0;
        visualizeButtonPressDuration();
    }

    if (isPressed) {
        currentDuration += ReadingTask::recurrenceDelay;
    }

}

void visualizeButtonPressDuration() {
    extern Led redLed;
    extern Led greenLed;

    if (ReadingTask::lastDuration < 500) {
        greenLed.on();
        redLed.off();
    } else {
        redLed.on();
        greenLed.off();
    }
}
```

Listing 3: ReadingTask.cpp

`StatisticsTask.cpp`

```
#include "StatisticsTask.h"
#include "ReadingTask.h"
#include <led/led.h>

// Global LED instances are defined in main.cpp
extern Led yellowLed;

namespace Statistics {
uint16_t shortPressesNumber = 0;
unsigned long shortPressesTotalDuration = 0;
```

```

uint16_t longPressesNumber = 0;
unsigned long longPressesTotalDuration = 0;

unsigned long lastHandledUpdateTime = -1;
} // namespace Statistics

void StatisticsTask::run() {

  // if there is a new update from the reading task, we update the statistics
  if (ReadingTask::lastUpdatedTime != Statistics::lastHandledUpdateTime) {
    Statistics::lastHandledUpdateTime = ReadingTask::lastUpdatedTime;

    if (ReadingTask::lastDuration > 0) {
      if (ReadingTask::lastDuration < 500 && ReadingTask::lastDuration > 0) {
        Statistics::shortPressesTotalDuration += ReadingTask::lastDuration;
        Statistics::shortPressesNumber++;
      } else {
        Statistics::longPressesTotalDuration += ReadingTask::lastDuration;
        Statistics::longPressesNumber++;
      }

      StatisticsLedTask::triggerBlinks(ReadingTask::lastDuration >= 500);
    }
  }
}

namespace StatisticsLedTask {

uint16_t recurrenceDelay = 2;
uint16_t offset = 2;
```

```cpp

// use the global yellowLed defined in main.cpp

// Internal state tracking
int blinksRemaining = 0;
unsigned long lastActionTime = 0;
const int blinkInterval = 100;

void triggerBlinks(bool isLong) {
  ::yellowLed.off();          // Force off immediately
  lastActionTime = millis(); // Reset timer so it blinks 'on' instantly
  blinksRemaining = isLong ? 10 : 5;
}

void run() {
  if (blinksRemaining <= 0)
    return;

  unsigned long currentTime = millis();
  if (currentTime - lastActionTime >= blinkInterval) {
    lastActionTime = currentTime;

    // Toggle logic
    if (::yellowLed.isOn()) {
      ::yellowLed.off();
      blinksRemaining--; // Decrease count after a full on/off cycle
    } else {
      ::yellowLed.on();
    }
  }
}
} // namespace StatisticsLedTask
```

Listing 4: StatisticsTask.cpp

`DisplayTask.cpp`

```cpp
#include <stdio.h>
#include "DisplayTask.h"
```

```cpp
#include "StatisticsTask.h"

void DisplayTask::run() {
    uint32_t totalCount = Statistics::shortPressesNumber + Statistics::longPressesNumber;
    uint32_t totalDuration = Statistics::shortPressesTotalDuration + Statistics::longPressesTotalDuration;

    if (totalCount > 0) {
        // Calculate average duration in pure milliseconds
        uint32_t avgMs = totalDuration / totalCount;

        // Extract whole seconds and the fractional remainder
        uint32_t wholeSeconds = avgMs / 1000;
        uint32_t fractionalSeconds = avgMs % 1000;

        // To get exactly 2 decimal places (e.g., .25s), divide the remainder by 10
        uint32_t decimals = fractionalSeconds / 10;

        // Use %lu (unsigned long) to ensure compatibility across all architectures
        printf("L: %u, S: %u, Avg: %lu.%02lus\n",
               Statistics::longPressesNumber,
               Statistics::shortPressesNumber,
               (unsigned long)wholeSeconds,
               (unsigned long)decimals);
    } else {
        // Fallback for before any buttons are pressed
        printf("L: %u, S: %u, Avg: 0.00s\n",
               Statistics::longPressesNumber,
               Statistics::shortPressesNumber);
    }

    // reset statistics after displaying
    Statistics::shortPressesNumber = Statistics::longPressesNumber = 0;
```

```
    Statistics::shortPressesTotalDuration = Statistics::
        longPressesTotalDuration = 0;

}
```

Listing 5: DisplayTask.cpp